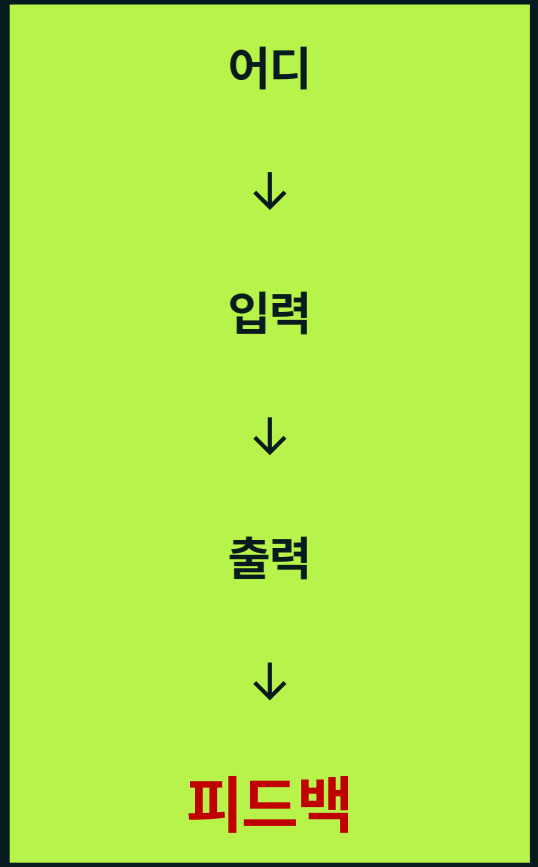
# K-Auto KG 실습
# E2E 따라하기 피드백

어디 · 입력 · 출력 · 피드백 — 25단계를 처음부터 끝까지




**K-Auto KG Stage 1 · 강사 · 수강생 공용**

# 한 단계는 어디 · 입력 · 출력 · 피드백 네 칸으로 읽습니다

예: STEP 07 — Neo4j Browser에서 RETURN 1을 쳐서 1이 나오면 접속 성공이다

| ① 어디 | ② 입력 | ③ 출력 | ④ 피드백 |
|---|---|---|---|
| 어느 화면에서 | 무엇을 쳤나 | 무엇이 나왔나 | 무엇을 말해 주나 |
| Neo4j Browser<br>localhost:7474 | `RETURN 1 AS ok;` | ok<br>1 | ✓ 접속 성공<br>× 인증 실패 → D3 |

**수강생은 ①~③을 따라 하고, 강사는 ③을 보고 ④를 말해 줍니다**

# 피드백은 칭찬 · 지적 · 다음 지시 세 마디로 합니다

초보자에게는 '틀렸다'보다 '어디가 다르고 다음에 무엇을 칠지'가 필요하다

✓ 맞으면

## 근거를 짚어 칭찬

- 출력의 맞는 줄
- 'OK 줄이 보이네요'

× 이상하면

## 다른 곳을 짚기

- 기대와 다른 한 줄
- 'got 0이 문제예요'

→ 다음 지시

## 칠 것을 알려 주기

- 단계 번호 · D1~D6
- 'STEP 05로 가세요'

※ 슬라이드 노트에 각 단계의 프롬프트 전문(P0~P14)과 강사 멘트를 넣었다 — 발표자 보기에서 바로 복사한다

## 지적은 항상 출력 화면의 특정 줄을 가리키며 합니다

# 출력 칸의 값은 네 가지 출처 가운데 하나입니다

오른쪽 위 태그를 보면 '반드시 같아야 하는 값'인지 '모양만 비슷하면 되는 값'인지 알 수 있다

| 실측 | CSV 계산 | 코드 확인 | 예시 |
| --- | --- | --- | --- |
| **실제로 실행** | **숫자가 같아야 함** | **문구가 같아야 함** | **모양만 비슷** |
| 똑같이 나와야 함 | 정상이면 일치 | 스크립트 원문 | PC마다 다름 |

※ Docker·Neo4j 실행과 MCP 호출 화면은 이 자료 제작 환경에서 실행하지 않았다 — 해당 출력은 '예시' 또는 'CSV 계산'이다

## 'CSV 계산' 숫자가 다르면 그 자리에서 멈추고 원인을 찾습니다

# 실습 전체는 25단계 · 네 구간으로 이어집니다

A 준비·설치 → B 데이터·적재 → C 탐색·분석 → D MCP·제출 — 앞 단계 출력이 맞아야 다음 단계로 간다

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| A 준비 · 설치 | 01 폴더 열기 | 02 P0 진단 | 03 T1 대조 | 04 P1 계획 | 05 P2 .env | 06 P3 컨테이너 | 07 Browser 접속 |
| B 데이터 · 적재 | 08 P4 테스트 | 09 P5 온톨로지 | 10 P6 승인 | 11 T5 적재 | 12 P7 verify | 13 C1 세기 | |
| C 탐색 · 분석 | 14 P8 HTML | 15 관계 필터 | 16 C3 밸류체인 | 17 C4 모비스 | 18 C6 공급 | 19 C8 2홉 | 20 P10 검증 |
| D MCP · 제출 | 21 P11 등록 | 22 P12 질의 | 23 P13 개선 | 24 P14 추가 | 25 보고서 | | |

## 막히면 그 단계의 '× 이상하면' 칸부터 봅니다

# A

# 준비 · 설치

STEP 01 ~ 07 — 폴더를 열고, 진단하고, Neo4j에 접속한다

예시

# 첫 단계는 터미널을 프로젝트 폴더 안에서 여는 것입니다

왜: data · scripts 같은 상대경로가 kauto_kg_mvp 폴더에서만 맞는다

**① 어디** — 파일 탐색기·Finder → 터미널 (PowerShell · Terminal)

**② 입력 · 위치 확인 명령**

```
Get-Location     # Windows
pwd              # macOS
```

**③ 출력 · 예시**

```
C:\Users\kim\...\kauto_kg_mvp
/Users/kim/.../kauto_kg_mvp
```

**④ 피드백 · ✓ 맞으면**

마지막 폴더명이 kauto_kg_mvp입니다

**④ 피드백 · × 이상하면 → 지적 · 다음 지시**

상위 폴더에서 열었다
→ 폴더 안으로 cd 한 뒤 다시 확인

## 이후 모든 명령은 이 폴더에서 실행합니다

예시

# PO는 아무것도 설치하지 않고 현재 상태만 조사하게 합니다

왜: 이미 있는 프로그램을 다시 깔면 기존 설정과 데이터가 깨질 수 있다

**① 어디** — Claude Code 채팅창 — 터미널에서 claude 실행

**② 입력 · PO 요지 (전문은 노트)**

```
README · CLAUDE.md를 읽고
OS · Python · Docker · 포트 · MCP
상태를 표로 보고해줘.
지금은 설치·변경하지 마.
```

**③ 출력 · 예시 · 표**

| | | |
|---|---|---|
| Python | 준비됨 | py 3.x |
| Claude Code | 준비됨 | --version OK |
| Docker | 확인불가 | 엔진 꺼짐 |
| MCP | 미설치 | 0개 |

**④ 피드백 · ✓ 맞으면**

판정마다 근거 명령이 붙어 있습니다

**④ 피드백 · × 이상하면 → 지적 · 다음 지시**

근거 없이 '설치됨' · 설치 시작
→ 'PO만 다시: 아무것도 설치하지 마'

**'확인 불가'와 '미설치'를 구분했는지가 첫 지적 포인트입니다**

예시

# 버전 명령을 직접 쳐서 Claude의 보고와 대조합니다

왜: 에이전트 보고를 그대로 믿지 않고 한 번 확인하는 습관을 만든다

**① 어디**  **터미널 — Claude Code 보고와 직접 대조 (T1)**

**② 입력 · T1**

```
claude --version
docker --version
docker compose ps
claude mcp list
```

**③ 출력 · 예시 · 흔한 결과**

```
docker --version → 성공
compose ps → Cannot connect
  to the Docker daemon
mcp list → No MCP servers
```

**④ 피드백 · ✓ 맞으면**

P0 표와 같은 결과입니다

**④ 피드백 · × 이상하면 → 지적 · 다음 지시**

Cannot connect ... daemon
→ 'Docker Desktop을 켜고 다시' (D2)

**'No MCP servers'는 오류가 아니라 아직 등록 전이라는 정상 표시입니다**

예시

# 설치 계획은 승인이 필요한 지점에서 멈춰야 합니다

왜: 관리자 권한 · 로그인 · 비밀번호 입력은 수강생 본인이 해야 하는 일이다

**① 어디** Claude Code 채팅창

**② 입력 · P1 요지 (전문은 노트)**

P0 결과 기준, 없는 것만
공식 경로로 설치 계획.
승인이 필요하면 직전에 멈춤.
설치 전후 버전 재확인.

**③ 출력 · 예시**

1. Docker Desktop 설치 필요
2. 관리자 승인이 필요합니다
   → 여기서 멈춥니다
3. 설치 후 docker --version

**④ 피드백 · ✓ 맞으면**

멈추고 '직접 승인하세요'라고 안내

**④ 피드백 · × 이상하면 → 지적 · 다음 지시**

멈추지 않고 진행 · 재설치
→ Esc로 중단하고 P1 다시 보내기

**'멈춰줘'를 지켰는지가 승인 경계 교육의 핵심입니다**

코드 확인

# .env 비밀번호는 내가 편집기에서 직접 입력합니다

왜: 비밀번호가 채팅이나 로그에 남으면 다른 사람에게 노출된다 (P2 · T2)

**① 어디**  **터미널 → 메모장 · TextEdit (.env 파일)**

**② 입력 · T2 · Windows**

```
Copy-Item .env.example .env
notepad .env
# macOS: cp ... 후 open -e .env
```

**③ 출력 · .env에서 고칠 줄 하나**

```
NEO4J_PASSWORD=
  CHANGE_TO_A_LONG_UNIQUE_PASSWORD
→ 8자 이상, 나만 아는 비밀번호로
```

**④ 피드백 · ✓ 맞으면**

.env만 고쳤고 채팅에 비밀번호가 없습니다

**④ 피드백 · × 이상하면 → 지적 · 다음 지시**

안내문 그대로 저장
→ 다음 단계에서 비밀번호 오류가 난다

**비밀번호를 채팅창에 붙여 넣었다면 바로 바꾸게 합니다**

예시

# Neo4j 컨테이너가 켜지면 이름 · 포트 · 상태로 확인합니다

왜: Docker 안의 DB가 실제로 떠 있어야 다음 단계가 가능하다

**① 어디** — 터미널 — Docker로 Neo4j 시작 (T3)

**② 입력 · T3**

```
docker compose up -d
docker compose ps
```

**③ 출력 · 예시 (열 일부)**

```
kauto-neo4j   neo4j:2026.09.0
Up 2 minutes
127.0.0.1:7474 · 7687
```

**④ 피드백 · ✓ 맞으면**

이름 kauto-neo4j · 상태 Up

**④ 피드백 · × 이상하면 → 지적 · 다음 지시**

missing a value 오류
→ .env를 안 만들었다 (STEP 05로)

**첫 실행은 플러그인을 내려받느라 몇 분 걸릴 수 있습니다**

예시

# Browser에서 RETURN 1이 1을 돌려주면 접속 성공입니다

왜: DB에 로그인되는지만 먼저 확인한다 — 기업 데이터는 아직 없다

**① 어디**  **웹 브라우저 → Neo4j Browser · http://localhost:7474**

**② 입력 · 로그인 후 입력**

사용자 neo4j
비밀번호 .env의 값

```
RETURN 1 AS ok;
```

**③ 출력 · 결과 표**

```
ok
1
```

**④ 피드백 · ✓ 맞으면**

표에 1이 보입니다

**④ 피드백 · × 이상하면 → 지적 · 다음 지시**

Authentication failed
→ 예전 볼륨의 비밀번호 (D3) · 볼륨 삭제 금지

**접속 성공은 적재 성공이 아닙니다 — 아직 기업 노드는 0개입니다**

B

# 데이터 · 적재

STEP 08 ~ 13 — CSV를 검사하고, 승인받아 넣고, 숫자로 검증한다

실측

# DB에 넣기 전에 CSV 검사 6건부터 통과시킵니다

왜: 잘못된 데이터는 DB에 넣은 뒤 지우기 어렵다 — 파일 단계에서 먼저 거른다

**① 어디** Claude Code 채팅창 (직접 하려면 터미널 · T4)

**② 입력 · P4 요지 (전문은 노트)**

```
.venv 만들고 패키지 설치
unittest 6건 실행
PASS/FAIL 목록 보고
```

**③ 출력 · 실측 (줄임)**

```
test_... ... ok    (6줄)
----------------------
Ran 6 tests in 0.005s
OK
```

**④ 피드백 · ✓ 맞으면**

ok 6줄과 마지막 OK가 보입니다

**④ 피드백 · × 이상하면 → 지적 · 다음 지시**

ModuleNotFoundError
→ .venv의 Python으로 다시 실행 (D4)

**이 테스트는 CSV만 검사합니다 — Neo4j 연결과는 무관합니다**

CSV 계산

# 온톨로지 설명은 사실 · 분류 · 추정을 나눠 받습니다

왜: 그래프의 선을 모두 거래로 읽는 것이 초보자가 가장 많이 하는 실수다

**① 어디** Claude Code 채팅창

**② 입력 · P5 요지 (전문은 노트)**

CSV와 import_neo4j.py를 읽고
노드·관계를 CSV 열에 매핑.
현대모비스·포스코퓨처엠 비교.
사실/분류/추정은 다른 표로.

**③ 출력 · 핵심 숫자**

현대모비스
  역량 4 · 공급 관계 0
포스코퓨처엠
  역량 2 · 공급계약 3 · 기존공급 1

**④ 피드백 · ✓ 맞으면**

레이블 6개를 CSV 열과 짝지었습니다

**④ 피드백 · × 이상하면 → 지적 · 다음 지시**

'모비스는 공급 관계가 없다'
→ '시드에 없을 뿐, 거래가 없다는 뜻 아님'

**연습 질문 두 개(같은 그룹 = 소유? 같은 역량 = 납품?)의 답은 모두 '아니다'입니다**

CSV 계산

# 적재 승인 전에 무엇이 몇 개 생기는지 설명을 받습니다

왜: DB에 쓰는 작업은 되돌리기 어렵다 — 승인 전에 내용을 알아야 한다

**① 어디**  **Claude Code 채팅창 — 적재 직전 승인 단계**

**② 입력 · P6 요지 (전문은 노트)**

DB 주소 · 기존 데이터,
추가될 노드 · 관계, 백업,
MERGE 중복방지를 먼저 설명.
승인 후에만 적재.

**③ 출력 · 모범 설명의 숫자**

bolt://127.0.0.1:7687 · neo4j
노드 154 · 관계 299 추가
제약조건 6개 · 삭제 없음
→ '진행할까요?'

**④ 피드백 · ✓ 맞으면**

숫자를 말하고 승인을 기다립니다

**④ 피드백 · × 이상하면 → 지적 · 다음 지시**

설명 없이 바로 실행
→ 중단시키고 P6 다시 보내기

**MERGE라서 두 번 실행해도 노드 154 · 관계 299가 그대로입니다**

코드 확인

# 적재 스크립트가 성공하면 네 줄이 출력됩니다

왜: 출력의 숫자가 CSV 개수와 같아야 빠짐없이 들어간 것이다

**① 어디** — 터미널 — 승인 후 적재 실행 (T5)

**② 입력 · T5 · macOS**

```
.venv/bin/python
  scripts/import_neo4j.py
# Windows: .venv\Scripts\python.exe
```

**③ 출력 · 스크립트 print 원문 (줄임)**

```
... assertions imported: 42
IMPORT COMPLETED (...)
Input: 30 companies; 24 ...; 7 ...
Evidence: {...: 11, ...: 19}
```

**④ 피드백 · ✓ 맞으면**

42 · 30 · 24 · 7을 확인했습니다

**④ 피드백 · × 이상하면 → 지적 · 다음 지시**

Set NEO4J_PASSWORD ... first.
→ .env 비밀번호 미설정 (STEP 05로)

**11 / 19는 공식 페이지 확인 11곳 · 추가 검토 19곳이라는 뜻입니다**

CSV 계산

# verify가 PASS 줄을 출력해야 적재 검증이 끝납니다

왜: 사람이 눈으로 맞추는 대신 스크립트가 CSV와 자동으로 비교한다

**① 어디** — 터미널 — 적재 검증 (verify_neo4j.py)

**② 입력 · T5 · macOS**

```
.venv/bin/python
  scripts/verify_neo4j.py
```

**③ 출력 · 핵심 줄**

```
companies: 30 · stages: 7
source_nodes: 42
company_company_edges: 42
PASS: ... SUPPLIES edges = 0.
```

**④ 피드백 · ✓ 맞으면**

마지막 줄이 PASS입니다

**④ 피드백 · × 이상하면 → 지적 · 다음 지시**

expected 30, got 0
→ 다른 DB에 넣었다 (.env의 DATABASE)

**숫자가 맞아 보여도 PASS 줄이 없으면 FAIL입니다**

CSV 계산

# Browser에서 관계를 직접 세면 여덟 줄 · 합계 42가 나옵니다

왜: 스크립트 결과를 내 눈으로 한 번 더 확인한다

## ① 어디 — Neo4j Browser — 상단 입력창 (C1)

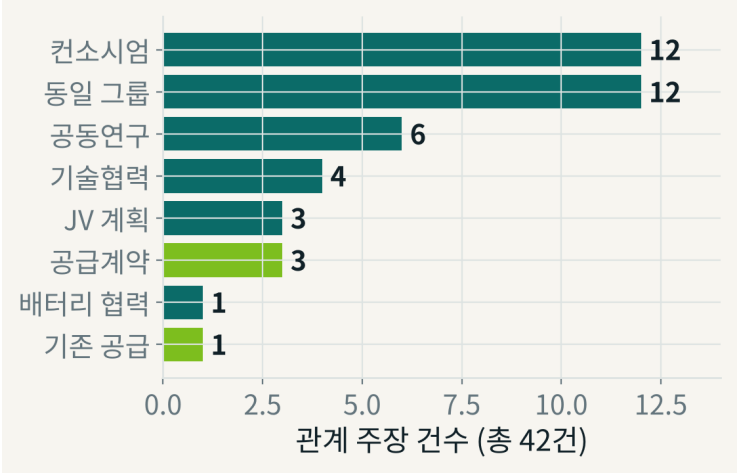
### ② 입력 · C1 (요지)

```
MATCH (:KAutoCompany)-[r]->
      (:KAutoCompany)
RETURN type(r), count(*)
```

✓ 8줄 · 합계 42

× 42보다 많다
→ 예전 데이터 (D6)

### ③ 출력 · 기대 결과 (표를 차트로)




**12건인 두 타입은 순서가 바뀌어 나와도 정답입니다**

C

# 탐색 · 분석

STEP 14 ~ 20 — 그래프를 보고, 질문하고, 선의 의미를 구분한다

실측

# 미리보기 HTML은 DB 없이도 그래프를 보여 줍니다

왜: Docker가 막힌 PC에서도 데이터 구조를 먼저 볼 수 있다 (Plan B)

**① 어디** 터미널 → 웹 브라우저 (output/kauto_graph.html)

**② 입력 · T6 · macOS**

```
.venv/bin/python
  scripts/render_html.py
open output/kauto_graph.html
```

✓ 상단 30·7·24·11·42

× 'DB에 적재됐다'
→ 아니다, CSV 미리보기

**③ 출력 · 실측 화면 (기본 선택 현대모비스)**




**이 화면은 Neo4j와 연결되어 있지 않습니다 — 적재 증거로 쓰지 않습니다**

실측

# 관계 필터를 바꾸면 선의 종류가 달라집니다

왜: 공급계약 · 공동연구 · 계열 관계를 눈으로 구분하는 연습이다

**① 어디** — 웹 브라우저 — 미리보기 화면 조작

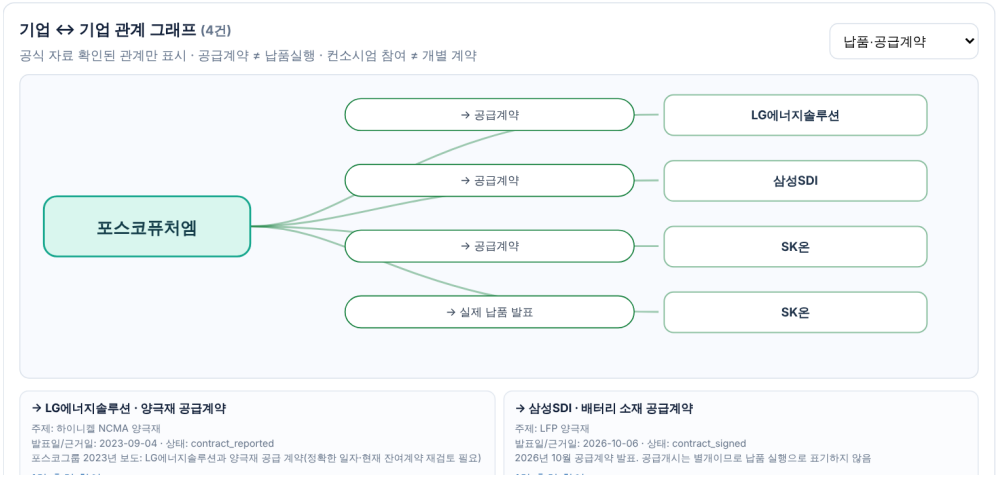
**② 입력 · 화면 조작**

검색: 포스코퓨처엠
관계 필터:
납품 · 공급계약

✓ 선 4개 · 카드 4장

× 현재 납품으로 읽음
→ 기존 공급 보도 1건

**③ 출력 · 실측 화면 — 4건**




**화면 라벨이 강해 보여도 근거는 rel_016 '기존 공급 보도' 한 건입니다**

CSV 계산

# 밸류체인별로 세면 일곱 줄 · 합계 30이 나옵니다

왜: 회사마다 대표 단계 하나만 배정한 학습용 분류를 확인한다

**① 어디** Neo4j Browser — 표 보기 (C3 · queries/01)

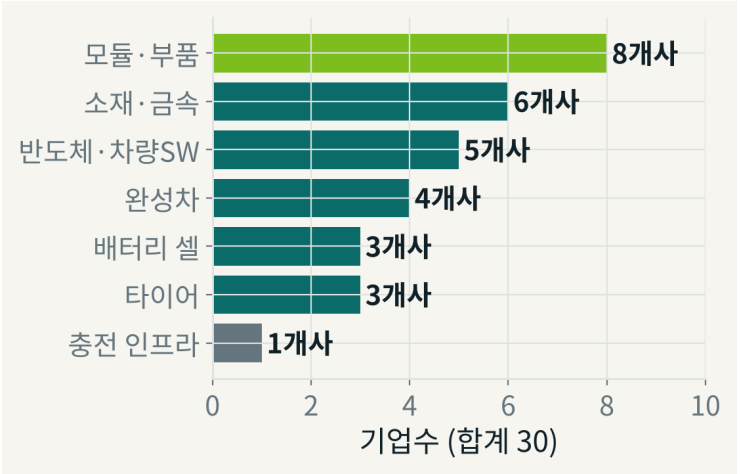
**② 입력 · C3 (요지)**

```
MATCH (c)-[:PARTICIPATES_
  IN_STAGE]->(s:KAutoStage)
RETURN s.name_ko, count(c)
```

✓ 7줄 · 합계 30

× 합계 30이 아님
→ 적재 누락 확인

**③ 출력 · 기대 결과 (표를 차트로)**




**한 회사가 여러 단계에 걸칠 수 있다는 점도 함께 말해 줍니다**

CSV 계산

# 현대모비스 그래프는 역량 4개와 시스템 3개로 나옵니다

왜: 역량(할 수 있는 것)과 거래(실제로 판 것)를 구분하는 연습이다

## ① 어디 — Neo4j Browser — 그래프 보기 (C4 · queries/02)

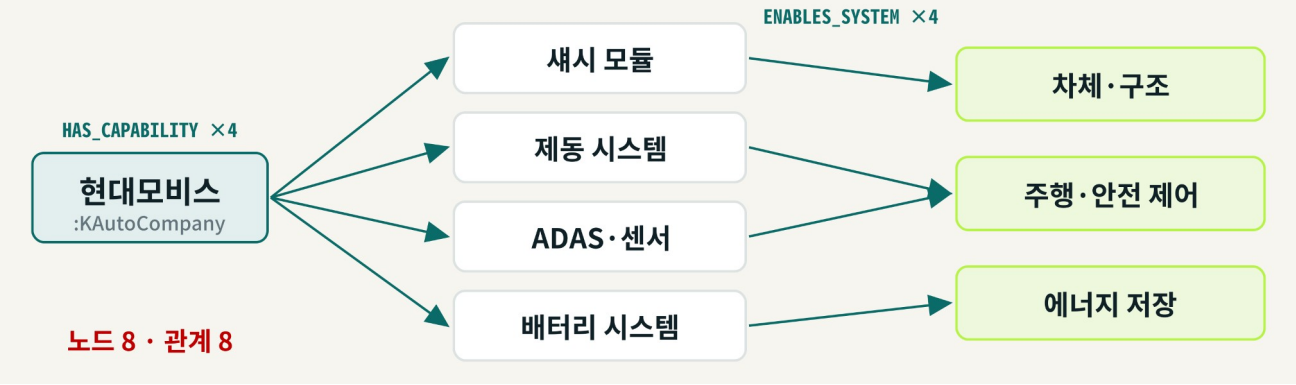
**② 입력 · C4 (요지)**

```
MATCH (c {id:'company.mobis'})
 -[:HAS_CAPABILITY]->(p)
 -[:ENABLES_SYSTEM]->(s)
```

✓ 역량 4 · 시스템 3

× '납품한다'고 읽음
→ 역량 분류일 뿐

**③ 출력 · 기대 그래프 — 노드 8 · 관계 8**




## C4 결과는 현대모비스의 고객사 목록이 아닙니다

CSV 계산

# 공급 관계를 뽑으면 계약 3건과 기존 공급 1건이 나옵니다

왜: '계약을 맺었다'와 '실제로 공급한다'는 서로 다른 사실이다

**① 어디**  **Neo4j Browser — 표 보기 (C6 · queries/08)**

**② 입력 · queries/08 (요지)**

```
MATCH (a)-[r:SUPPLIES_TO|
  SUPPLY_AGREEMENT_WITH]->(b)
RETURN a.name_ko, type(r),
       b.name_ko
```

**③ 출력 · 기대 결과 4행 (공급자 포스코퓨처엠)**

| | |
|---|---|
| AGREEMENT | LG에너지솔루션 |
| AGREEMENT | 삼성SDI |
| AGREEMENT | SK온 |
| SUPPLIES_TO | SK온 (음극재) |

**④ 피드백 · ✓ 맞으면**

4줄 · 계약 3 · 기존 공급 1

**④ 피드백 · × 이상하면 → 지적 · 다음 지시**

'네 곳에 납품 중'
→ 계약 3 · 납품 보도 1로 정정

**AGREEMENT는 SUPPLY_AGREEMENT_WITH를 줄여 적은 것입니다**

CSV 계산

# 2홉 경로는 연결을 보여 줄 뿐 충격 전파를 증명하지 않습니다

왜: '이어져 있다'와 '피해가 옮겨 간다'는 다른 주장이다

**① 어디** Neo4j Browser — 그래프 보기 (C8 · queries/09)

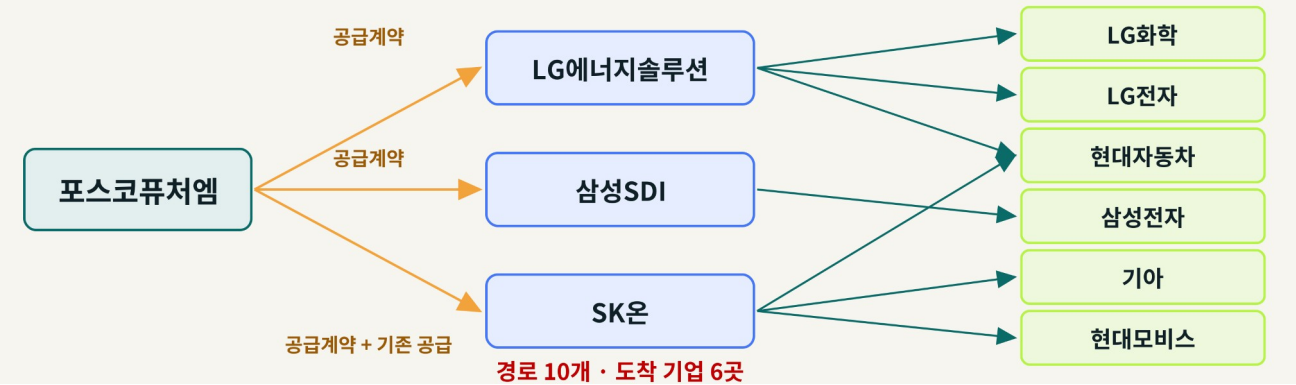
**② 입력 · C8 (요지)**

```
MATCH p=(o {id:'company.
  poscofuturem'})-[]-(mid)
  -[]-(dest) RETURN p
```

✓ 경로 10개 · 도착 6곳

× '매출이 줄어든다'
→ 의존도 정보 없음

**③ 출력 · 기대 그래프 — 경로 10 · 도착 6곳**




**SK온 경유 경로가 두 배인 것은 SK온까지 관계가 두 건이기 때문입니다**

CSV 계산

# 관계 검증 보고는 여섯 칸을 채우고 모르면 '미확인'으로 둡니다

왜: 근거가 없는 숫자(지분율 · 납품량)를 추정해 넣는 것을 막는다

**① 어디** Claude Code 채팅창

**② 입력 · P10 요지 (전문은 노트)**

```
두 기업의 관계를 (A)타입 (B)방향
(C)주제 (D)발표일 (E)상태
(F)출처 URL로 정리.
추정 금지 · 없으면 '미확인'.
```

**③ 출력 · 현대차 → 현대모비스**

```
AFFILIATED_WITH
  발표일 미상 · group_membership
CO_MEMBER_OF_CONSORTIUM
  2023-04-07 · MBD 컨소시엄
```

**④ 피드백 · ✓ 맞으면**

여섯 칸이 모두 채워졌습니다

**④ 피드백 · × 이상하면 → 지적 · 다음 지시**

'모비스 지분 ○%' 추정
→ 0점, 공시 없이 쓰지 않는다

**포스코퓨처엠–현대차를 고르면 직접 관계 0건 — '미확인'이 정답입니다**

D

# MCP · 제출

STEP 21 ~ 25 — Claude가 DB를 직접 읽게 하고, 보고서로 마무리한다

예시

# MCP 등록은 목록에서 Connected가 보여야 끝납니다

왜: Claude Code가 DB를 읽기 전용으로 직접 조회하는 통로를 연다

**① 어디** Claude Code 채팅창 → 터미널 (claude mcp list)

**② 입력 · P11 요지 (전문은 노트)**

```
mcp_launcher.py를 read-only로
절대 경로로 등록, 중복 금지.
비밀번호는 .env에서만.
→ claude mcp list
```

**③ 출력 · 예시**

```
neo4j-kauto:
  .../.venv/bin/python
  .../scripts/mcp_launcher.py
  - ✓ Connected
```

**④ 피드백 · ✓ 맞으면**

✓ Connected가 보입니다

**④ 피드백 · × 이상하면 → 지적 · 다음 지시**

Failed to connect
→ .env 비밀번호 · 경로 확인 (D5)

**등록 뒤에는 Claude Code 세션을 다시 시작해야 인식될 수 있습니다**

CSV 계산

# MCP 질의는 도구 이름과 결과를 함께 보여 줘야 성공입니다

왜: 등록됐다고 해서 실제로 호출되는 것은 아니다

**① 어디** — Claude Code 채팅창 — MCP 도구로만 조회

**② 입력 · P12 요지 (전문은 노트)**

neo4j-kauto MCP 도구로만
레이블 · 기업 수 · 관계 타입 수 ·
모비스 역량을 조회.
사용한 도구명도 보여줘.

**③ 출력 · 기대 값**

레이블: KAuto 6종 + 공용 5종
기업 노드: 30
관계: 8종 · 합계 42
모비스: 섀시·제동·ADAS·배터리

**④ 피드백 · ✓ 맞으면**

사용한 MCP 도구 이름이 적혀 있습니다

**④ 피드백 · × 이상하면 → 지적 · 다음 지시**

Python 조회로 'MCP 성공'
→ MCP 항목을 FAIL로

**등록됨과 호출 성공은 서로 다른 칸에 적습니다**

실측

# 개선 요청에는 수정 전 설계와 테스트 계획을 먼저 받습니다

왜: 이미 있는 기능(주제 · 발표일 · 상태 · URL)을 다시 만들지 않기 위해서다

**① 어디** — Claude Code 채팅창 — 수정 전 설계 단계

**② 입력 · P13 요지 (전문은 노트)**

관계를 누르면 6개 항목이
보이도록 개선안 제시.
바뀔 파일과 테스트 계획 먼저.
출처 없는 관계 금지.

**③ 출력 · 모범 설계안**

HTML 카드에 두 항목만 추가
  relation_type · reviewed_on
Cypher C7에 reviewed_on 열
테스트 6건 + HTML 재생성

**④ 피드백 · ✓ 맞으면**

바뀔 파일을 먼저 말합니다

**④ 피드백 · × 이상하면 → 지적 · 다음 지시**

설계 없이 바로 파일 수정
→ 멈추고 '설계부터 보여줘'

**실측 HTML 카드에는 주제 · 발표일 · 상태 · 요약 · 출처가 이미 있습니다**

코드 확인

# 새 기업이나 최근 관계를 추가하면 테스트가 FAIL할 수 있습니다

왜: 테스트에 '기업 30개'와 '발표일 ≤ 2026-10-19'가 고정돼 있다

**① 어디** — Claude Code 채팅창 — 관계 1건 추가 연습 (선택)

**② 입력 · P14 요지 (전문은 노트)**

```
공식 1차 출처부터 찾고
허용 타입 검사 → diff 제안.
승인 전 저장 금지.
승인 후 테스트 · HTML · 적재.
```

**③ 출력 · 흔한 FAIL**

```
company_count_unique_ids
  FAIL (기업 31 ≠ 30)
schema_and_provenance
  FAIL (발표일 상한 초과)
```

**④ 피드백 · ✓ 맞으면**

출처 → diff → 승인 → 테스트 순서

**④ 피드백 · × 이상하면 → 지적 · 다음 지시**

FAIL을 없애려 테스트부터 수정
→ 승인받고 이유를 기록

## 초보 수강생에게는 기존 기업 사이 관계 1건 추가만 권합니다

예시

# 보고서는 확인된 사실과 아직 증명되지 않은 사실을 나눠 적습니다

왜: '모른다'를 적는 것이 이 실습에서 가장 중요한 결과다

**① 어디**  **편집기 · Obsidian — student_report.md**

**② 입력 · 작성**

student_report_TEMPLATE.md
복사해 작성
(비밀번호 · 토큰은 넣지 않음)

**③ 출력 · 모범 예 · 포스코퓨처엠 → SK온**

확인됨: LFP 공급계약 발표
(2026-09-22, 2027~2029년 공급)
미확인: 납품 개시 · 물량 ·
매출 인식 시점

**④ 피드백 · ✓ 맞으면**

두 칸이 모두 채워졌습니다

**④ 피드백 · × 이상하면 → 지적 · 다음 지시**

'미확인' 칸이 비어 있음
→ 아직 모르는 것을 적게 한다

**실패를 만나고 원인 → 수정 → 재시험을 남긴 보고서에 보너스를 줍니다**

# 막힌 화면은 여섯 가지 진단 중 하나로 보냅니다

D1~D6 — 원인을 구분하는 확인 명령부터, 기존 데이터는 건드리지 않는다 (진단 프롬프트 전문은 노트)

D1 · 터미널

**claude 없음**

설치 vs PATH 구분

D2 · 터미널

**Docker 연결 실패**

Desktop 실행 · WSL2

D3 · Browser

**인증 실패**

원래 비번 · 볼륨 보존

D4 · 터미널

**ModuleNotFound**

.venv Python 직접 호출

D5 · 채팅

**MCP 호출 안 됨**

경로 · 패키지 · .env

D6 · Browser

**42건이 아님**

읽기 쿼리로만 점검

**어떤 경우에도 볼륨 삭제나 DETACH DELETE는 해법이 아닙니다**

어디서 무엇을 쳤는지 남기고

출력의 한 줄을 짚어 말하고

**모르는 것은 미확인으로 둔다**

---

피드백은 화면의 한 줄과
다음에 칠 명령입니다.

※ CSV 계산값은 시드 데이터(출처 검토일 2026-10-19) 기준. Docker·Neo4j·MCP 화면은 이 자료 제작 환경에서 실행하지 않았으며 '예시'로 표시했다.